



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

CSCC 2026

全国大学生计算机系统能力大赛 操作系统设计赛内核赛道

TxKernel

Rust 实现的类 *Unix* 多核异步操作系统

TxKernel开发团队

队员：杨岩琰、刘佳硕、孟书培 指导教师：夏文、仇洁婷

主讲人：杨岩琰

TxKernel

比赛成绩

2026.06.30 初赛排行榜12名

10	T2026103369910030	拼好核	杭州电子科技大学	233	2026-06-29 11:59:47	102.0	102.0	102.0	102.0	54.0	54.0	53.0	54.0
11	T202610486999642	最近一次入门Rust	武汉大学	91	2026-06-29 09:38:23	102.0	99.0	102.0	102.0	54.0	54.0	54.0	54.0
12	T2026181239910991	老被炸	哈尔滨工业大学(深圳)	45	2026-06-18 13:29:08	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
13	T202610486999803	AAAAAAAAAAAAAAAAAAAA	武汉大学	72									

得分详情

测试点	glibc-la	glibc-rv	musl-la	musl-rv			总分
basic	102	102	102	102			408
busybox	54	54	54	54			216
cyclictst	4.0	4.089151450053706	0.0	4.0			12.089151450053706
iozone	20.0	20.0	20.12780331920616	20.0			80.12780331920615
iperf	6.0	6.0	6.0	6.0			24.0
libcbench	31.86815518146383	29.425182957126605	29.37814856794794	27.35490599531117			118.02639270184955
libctest	-	-	214	220			434
lmbench	42.74867469736335	41.2294577336661	42.41515345700102	41.264337887629964			167.65762377566043
ltp	6193	6181	6116	6130			24620
lua	9	9	9	9			36
netperf	5.0	5.0	5.0	4.0			19.0
总分	6467.616829878827	6451.743792140846	6597.921105344155	6617.619243882941			2514.9009712467696

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

完成情况

项目总览



Rust语言开发



243条标准系统调用



本地通过初赛所有测例
各平台LTP 6000+分



Busybox, vi, tcc,
sqlite3等现实程序支持



FAT32&ext4文件系统支持



采用异步无栈协程调度

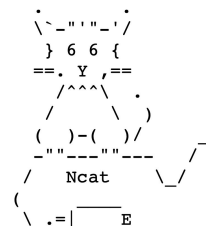
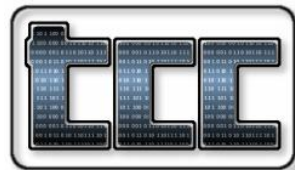


网络支持



自主设计异步profile框架

支持用户态应用



系统介绍

未来计划

AI使用声明

总结

TxKernel



完成情况

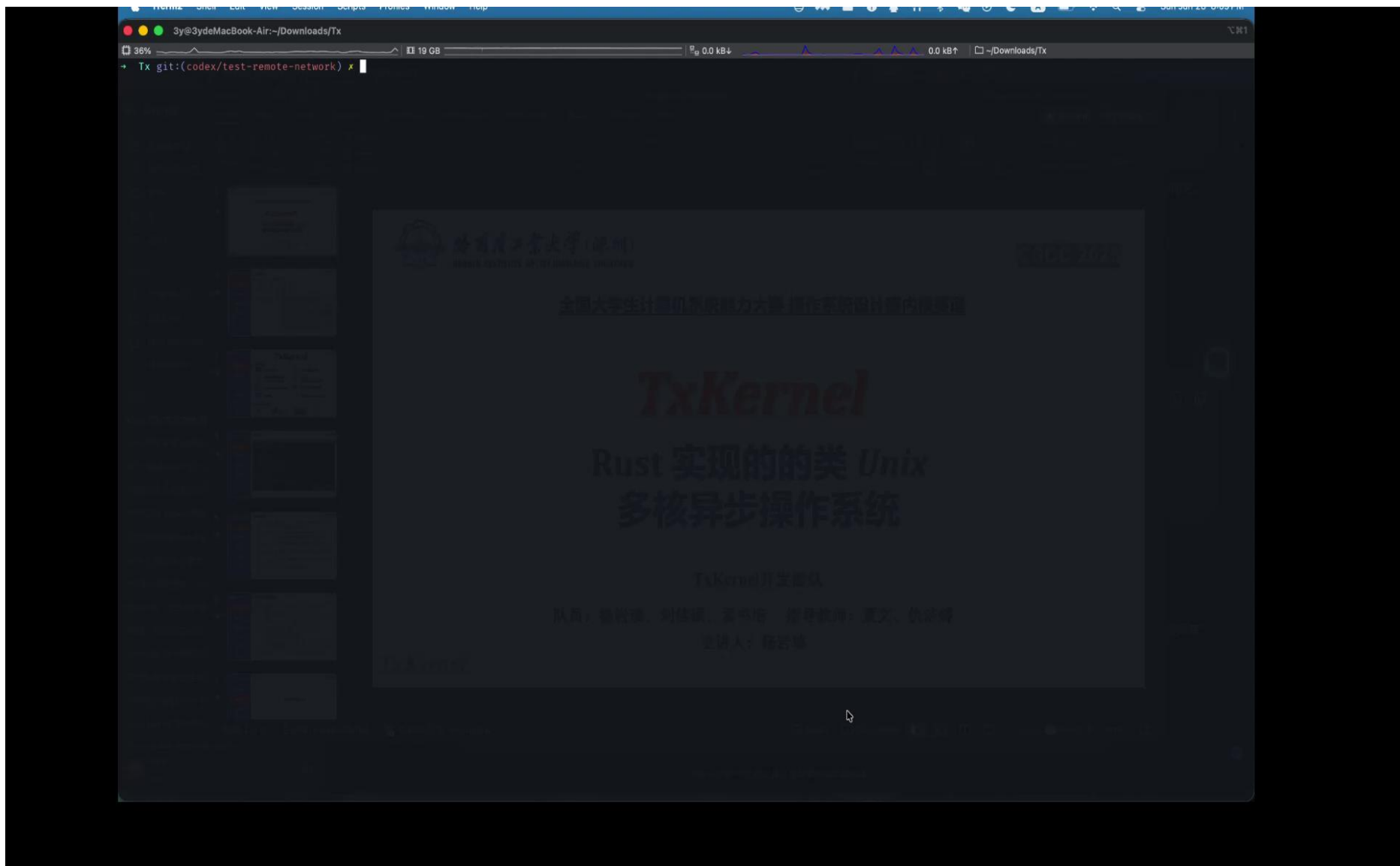
系统介绍

未来计划

AI使用声明

总结

TxKernel





项目历程

完成情况

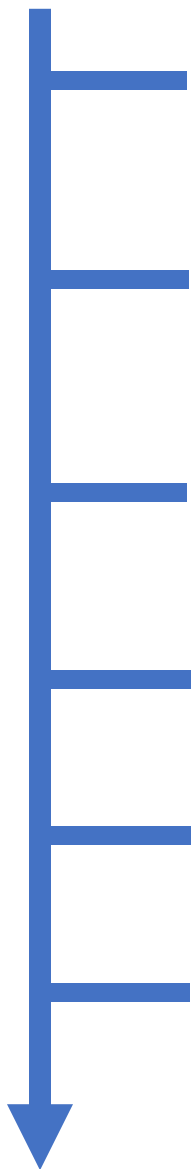
系统介绍

未来计划

AI使用声明

总结

TxKernel



1月

利用AI学习Rust, 完成rCore实验, 参考历年作品及FreeBSD
决定借鉴 Chronix 无栈协程架构

2月

基于FreeBSD子系统架构及无栈协程架构尝试第一版内核
实现; 但发现其对伪文件系统支持不够原生, 遂放弃

3月

学习 Linux 对象模型, 总结第一版内核经验设计第二版内核

4月

迁移已有代码, 实现 reactor、内存管理及硬件层, xtask 工具

5月

基于设计文档实现线程、文件系统、pthread 及 profiling 框架

6月

LTP 适配、性能调试及真实程序兼容性

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

功能域	数量	大概描述
进程/线程/ 命名空间/ 调度	41	负责程序生命周期和执行上下文：创建进程/线程、 <code>execve</code> 装载程序、 <code>wait4</code> 等待子进程、退出、进程组/session、namespace 切换、 <code>pidfd</code> 、调度参数查询/设置等。
VFS 路径/ 元数据/挂载	40	负责路径解析、目录项、文件元数据和挂载： <code>openat</code> 、 <code>statx</code> 、 <code>getdents64</code> 、 <code>mkdirat/unlinkat/renameat2</code> 、权限/属主修改、 <code>mount/umount2</code> 、 <code>sync/fsync</code> 等。
IPC/事件/ 异步 IO	39	负责进程间通信和事件等待：SysV shm/msg/sem、POSIX mq、 <code>epoll</code> 、 <code>eventfd</code> 、 <code>signalfd</code> 、 <code>inotify/fanotify</code> scaffold、Linux AIO 和部分 <code>io_uring</code> 。
基础文件 IO/fd/ 管道/splice	27	负责普通 fd 数据读写和 fd 操作： <code>read/write</code> 、vector IO、 <code>pread/pwrite</code> 、 <code>fcntl</code> 、 <code>dup/close</code> 、 <code>pipe2</code> ，以及 <code>sendfile/splice/tee/vmsplice</code> 这类零拷贝/管道转发接口。
凭证/权限/ 资源限制	24	负责用户/组身份、权限与资源限制： <code>setuid/setgid</code> 系列、 <code>getuid/getgid</code> 、groups、capability 查询/设置、 <code>nice/io</code> priority、 <code>getrlimit/setrlimit/prlimit64</code> 、 <code>getrandom</code> 。
内存/VM	19	负责用户地址空间和内存映射： <code>brk</code> 、 <code>mmap/munmap/mprotect/mremap</code> 、 <code>msync</code> 、锁页接口、 <code>mincore/madvise</code> 、 <code>userfaultfd</code> 、 <code>memfd_create</code> 等。
网络/socket	18	负责 socket ABI：创建/绑定/监听/连接/接受连接、收发数据、 <code>sendmsg/recvmmsg</code> 批量消息接口、 <code>setsockopt/getsockopt</code> 、 <code>shutdown</code> 、 <code>socketpair</code> 。
时间/定时器	17	负责时间读取、睡眠和定时器： <code>clock_gettime/clock_settime/clock_getres</code> 、 <code>nanosleep/clock_nanosleep</code> 、 <code>gettimeofday/settimeofday</code> 、 <code>timerfd</code> 、POSIX <code>timer_*</code> 、 <code>getitimer/setitimer</code> 。
信号/futex	13	负责异步信号和线程同步： <code>rt_sigaction</code> 、 <code>rt_sigprocmask</code> 、 <code>rt_sigreturn</code> 、 <code>rt_sigtimedwait</code> 、 <code>sigaltstack</code> 、 <code>kill/tkill/tgkill</code> ，以及 pthread/glibc/musl 依赖很重的 <code>futex</code> 。
安全/观测/ 项目私有	5	一部分是大型安全/观测接口的入口，如 <code>bpf</code> 、 <code>perf_event_open</code> ；另一部分是 txKernel 自己的 <code>tx_observe_*</code> tracing 控制 syscall。

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

系统特色介绍

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

利用异步无栈协程
统一 订阅-阻塞-唤醒 模型



完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

► rCore如何处理？

- 将本线程放入任务队列的末端
- 轮询开销

► XV6如何处理？

- Sleep(chan,lock) & wakeup(chan)
- 遍历线程列表，惊群效应
- 全局大锁

► Linux如何处理？

- waitQueue / io_uring
- 逐年积累，工程复杂度高
- 每个子系统自己实现阻塞机制

► Tx如何处理？

- Mailbox实现多对多订阅+事件筛选
- 精准投递
- 子系统统一使用

完成情况

系统介绍

未来计划

AI使用声明

总结

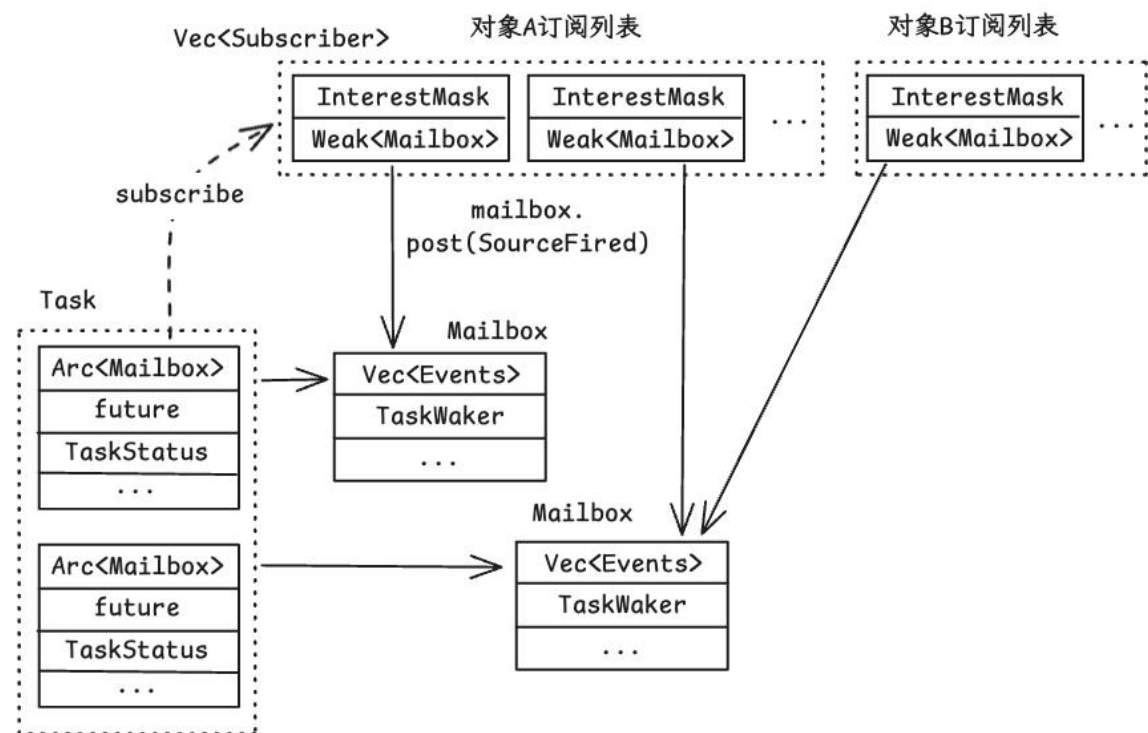
TxKernel

► Future 拥有自己的 Mailbox

- 直接向内核对象注册
- 内核对象根据兴趣掩码投递
- Reactor处理唤醒逻辑
- 拓展性强于历届硬编码设计
- 解决“怎么醒”

► 建模内核所有异步事件

- `SourceFired` 对象自定义事件
- `AgentReplied` 代理执行完毕 (ptrace预留)
- `AgentReplied` 信号
- `TimerFired` 计时器



```
pub enum MailboxEvent {
    SourceFired { /* ... */ },
    AgentReplied { /* ... */ },
    Abort { /* ... */ },
    SignalDelivered { /* ... */ },
    TimerFired { /* ... */ },
}
```

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

► 统一的阻塞语言

- 两个类型建模所有系统调用的阻塞情况 + 阻塞原因
- `Read` 被中断、`fanotify` 等待文件变化、`writew` 分步完成、`epoll` 等复杂逻辑自然被建模
- 只提醒不传值，醒来统一重新检查
- 系统调用实现时只提供同步step
- “怎么停，停下时订阅什么，怎么醒”都可以统一处理

► 统一的阻塞处理

- 一个match函数，成为所有系统调用的通用入口

```
pub enum StepOutcome<T, P> {  
    Continue { progress: P },  
    Yield { progress: P, shape: YieldShape },  
    Done(T),  
    Err(errno),  
}
```

```
pub enum YieldShape {  
    // 等待对象就绪，例如 pipe socket  
    OnWaitSource {  
        source: WaitSourceId,  
        interests: InterestMask,  
    },  
    // 等待另一task以自己的权限执行，为ptrace预留  
    OnAgent {  
        endpoint: DelegateEndpoint,  
        request: DelegateRequest,  
        token: DelegateToken,  
        deadline: Deadline,  
        cancel: AgentCancelPolicy,  
    },  
    // 等待计时器  
    OnTimer {  
        token: TimerId,  
        deadline: Deadline,  
    },  
    // 等待epoll子系统实现的通知  
    OnEdge {  
        source: WaitSourceId,  
        interests: InterestMask,  
    },  
}
```

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

统一的 Profiler 框架 Tx-observe

完成情况

系统介绍

未来计划

AI使用声明

总结

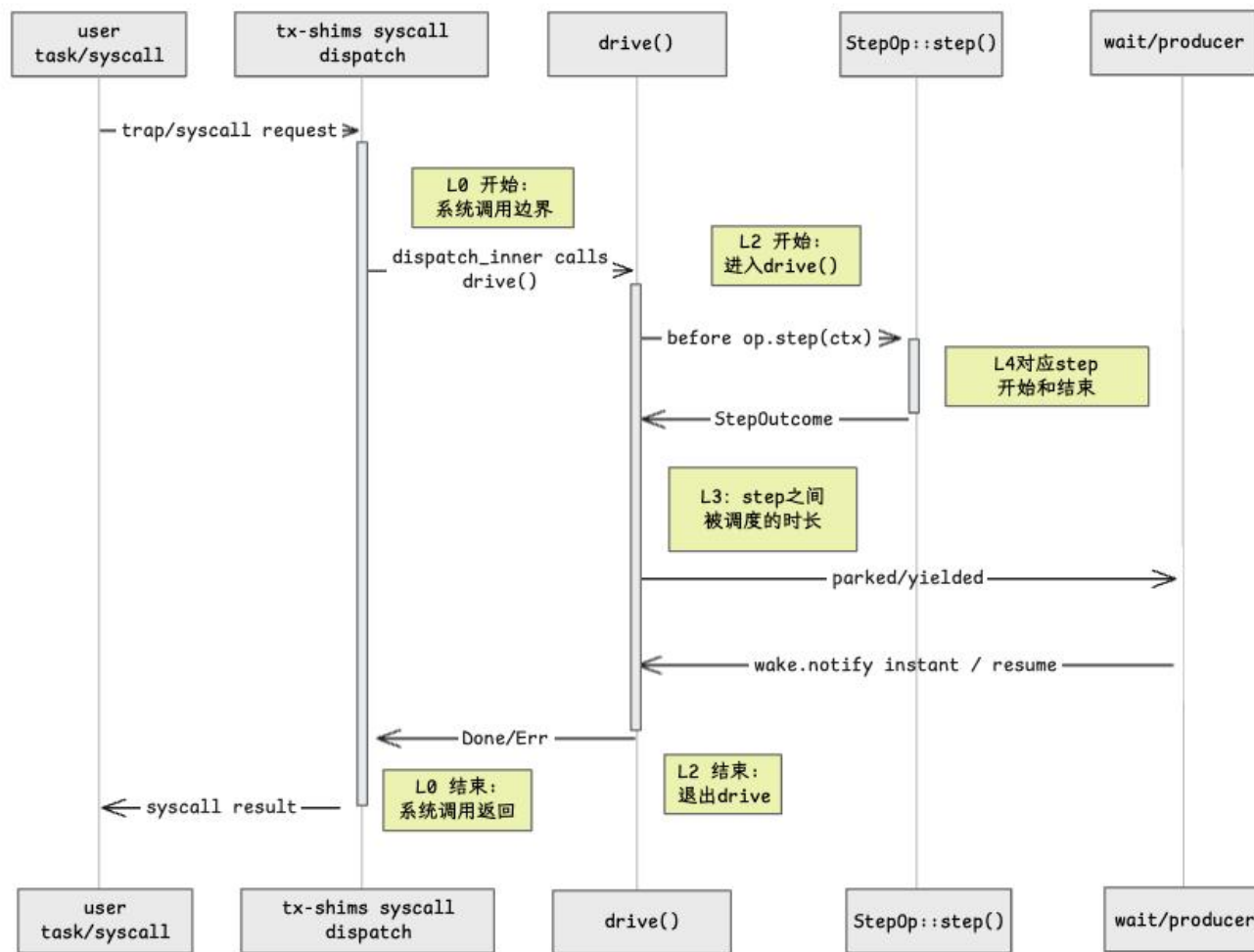
TxKernel

► 面向阻塞建模+协作式调度的优点

- 每一次yield都回到统一的代码边界，天然的计时点！
- 在统一的阻塞入口打印时间戳，自动得到所有系统调用的运行统计信息
- 基于边界span
- 亦可自定义marker

► 粒度细至每一poll

- 系统调用总时长 ✓
- 每一步step时长 ✓
- 阻塞时长 ✓
- 异步执行总时长 ✓



完成情况

系统介绍

未来计划

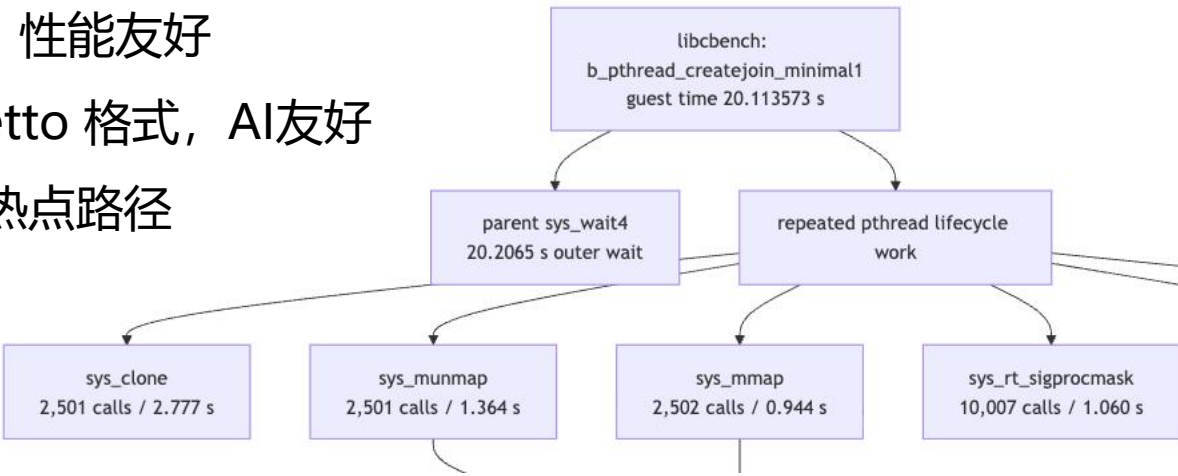
AI使用声明

总结

TxKernel

► 解决异步内核难以调试/调优问题

- 利用 qemu ivshmem 和宿主机进程共享内存
- 无分配 & 无锁打印调度日志，性能友好
- 支持导出至 parquet 和 perfetto 格式，AI友好
- SQL脚本自动导出百分位数+热点路径



Rank	Span	Count	Total	p50	p99	Max	Interpretation
0	sys_wait4	1	20.206500 s	20.206500 s	20.206500 s	20.206500 s	outer benchmark wait window
1	sys_clone	2,501	2.777393 s	1.059 ms	1.950 ms	16.682 ms	pthread thread creation and task setup
2	sys_munmap	2,501	1.363769 s	529 us	932 us	2.961 ms	pthread stack / mapping teardown
3	sys_futex	5,007	1.249112 s	260 us	666 us	12.594 ms	join / wake / clear-child-tid synchronization
4	sys_rt_sigprocmask	10,007	1.060409 s	95 us	218 us	1.321 ms	musl pthread signal-mask choreography
5	sys_mmap	2,502	943.696 ms	362 us	702 us	5.222 ms	pthread stack / TLS mapping setup
6	sys_exit	2,500	832.667 ms	316 us	636 us	1.998 ms	thread exit publication and cleanup

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

内核对象 身份与载荷**分离**设计

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

- ▶ 内核通过一系列索引暴露内核对象
 - 索引与实际后端解耦
- ▶ 内核对象身份与实际载荷生命周期不同
 - 进程 zombie 后身份待 reap
 - 但已经不可操作 (返回ESRCH)
- Inode同时承担路径索引、页容器、延迟绑定等职责
- 但伪文件系统只想要路径节点身份
- Linux结构体定义长达 100 行
- 污染缓存

完成情况

系统介绍

未来计划

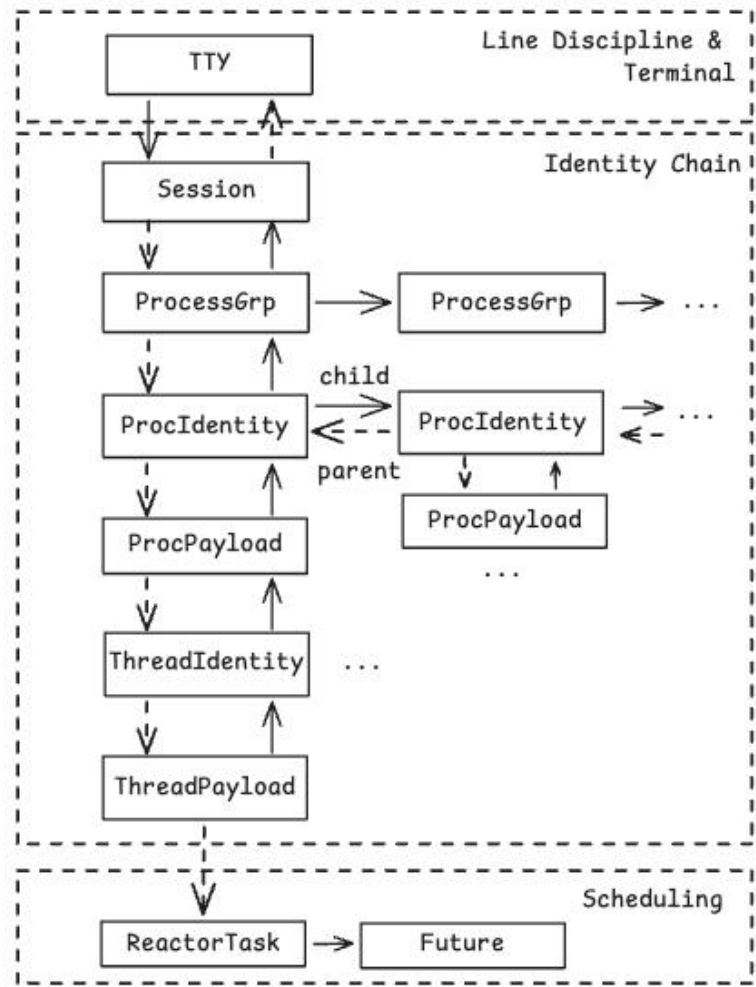
AI使用声明

总结

TxKernel

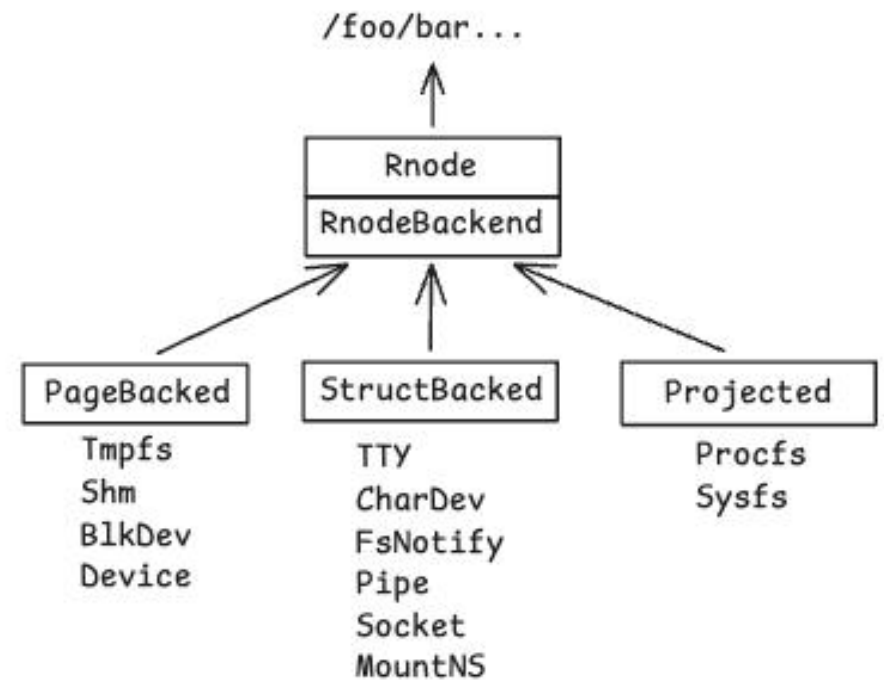
TCB -> Identity + Payload

- 身份 - 作业控制
- 载荷 - 执行资源+上下文



Inode -> Rnode + RnodeBackend泛型

- 身份 - 轻量级 VFS walk 节点
- 载荷 - 自己管理所属数据
- 简化各类文件系统实现



完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

身份与载荷分离设计——优点

► 更清楚的POSIX语义

- 直接用载荷存在状态表示生命周期

► 更高缓存命中率

- 身份解析时只加载 Identity, Payload 按需加载
- 简化RCU实现

► 更有效的生命周期管理

- 身份读多写少 -> EBR回收 + RCU无锁读
- 载荷写多读少 -> 引用计数, 但无关读更少
- 基于此实现的智能指针:
 - Weak - 持有者观察后升级, 防止ABA
 - IdentRef - 升级后结果, 在EBR Epoch内有效
 - Cap - 引用计数 Pin
- 将并发纪律用类型表示, 与对象模型配合

完成情况

系统介绍

未来计划

AI使用声明

总结

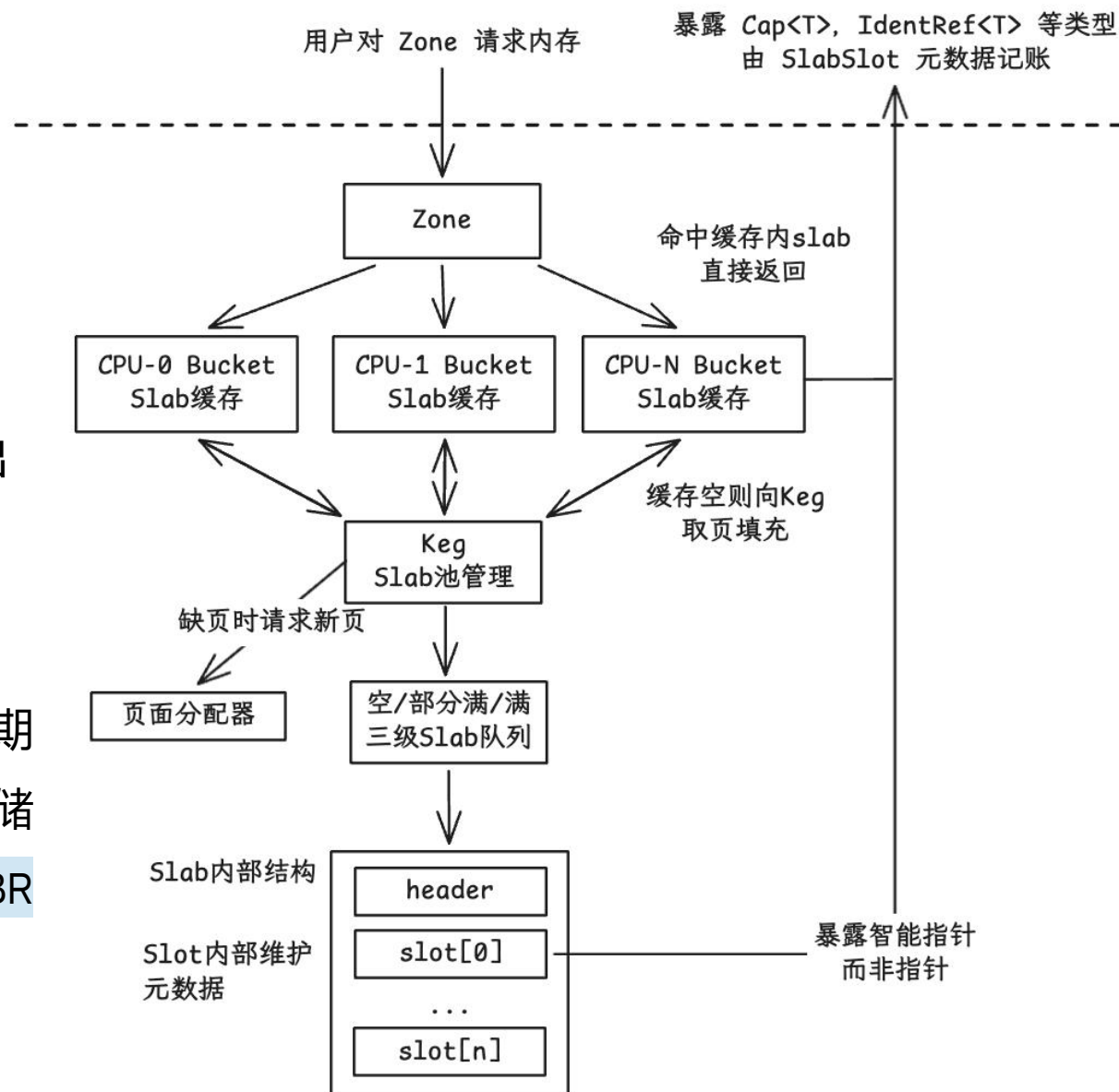
TxKernel

参考FreeBSD对象池设计

- 职责清晰分层：
 - Zone管语义
 - Bucket管缓存
 - Keg管页面池
- 调度逻辑简化, 不考虑换出

统一生命周期管理

- 内核对象均通过对象池分配
- 统一用智能指针管理生命周期
- 内核对象身份和载荷分池存储
- 便于内核对象身份部分的 EBR / RCU 实现



完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

其他功能模块与细节

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

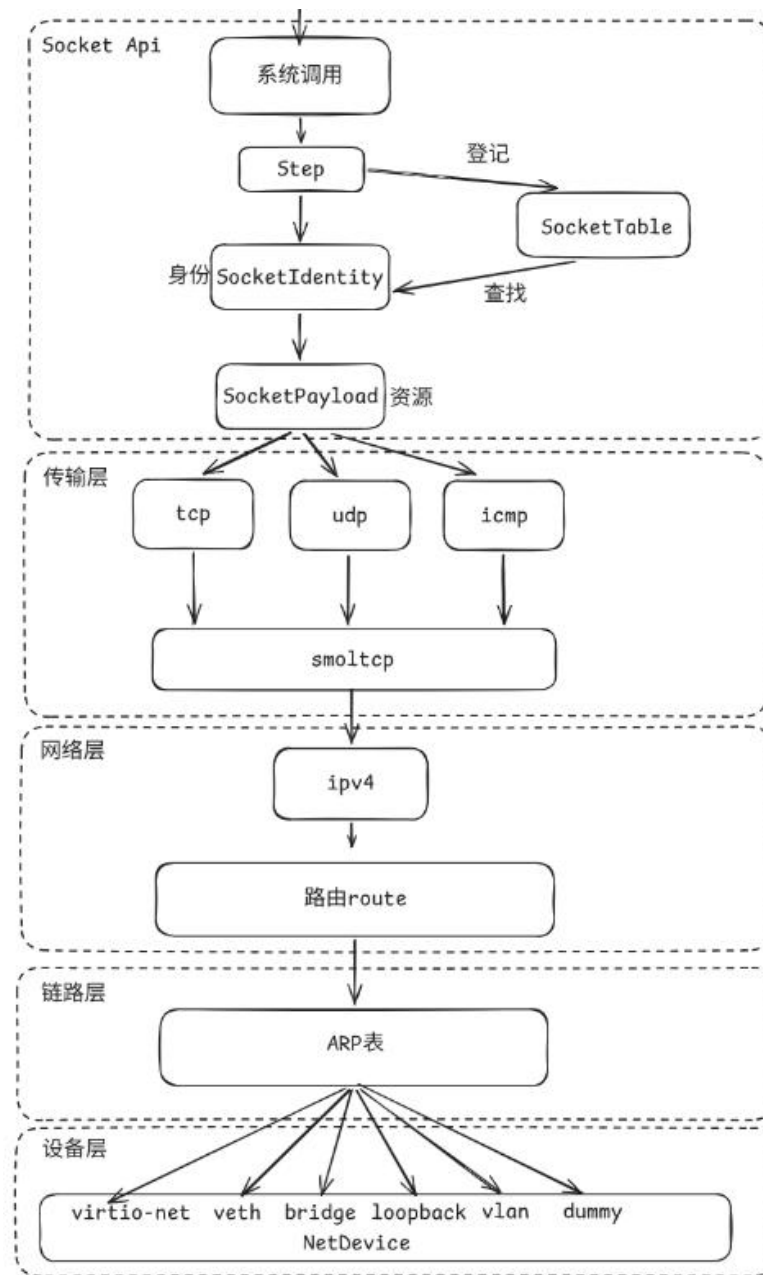
网络栈

► 协议覆盖:

- 实现了 TCP、UDP、ICMP
- 对上提供 POSIX 兼容的 BSD socket 接口
- 对下驱动 virtio-net 网卡和本地回环等设备

► 核心设计:

- Socket状态机和身份分离设计
- 利用异步架构适配smol-tcp crate状态机
- 网络设备抽象层对接6大设备:
 - loopback、virtio-net、veth、bridge、vlan 与 dummy



完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

▶ 基于区间树的细粒度锁

- $O(\log N)$ 细粒度查询

▶ ProcFS支持

- `proc/<pid>/{mem, maps}`

▶ 内核与用户共用页表

- 指令集后发优势, 无 KPTI 需求

▶ pre-fault处理

- 系统调用一旦发起对内核对象的修改, 不允许失败, 必须执行完毕
- 在执行前确保所有资源存在, 在准备阶段提前访问用户数据分配页面

代码 LOAD
数据 LOAD
BSS
用户栈 +
信号trapframe

```
/ # cat proc/1/maps
10000-109000 r-xp 00000000 00:00 0 [anon]
109000-10a000 rw-p 000f8000 00:00 0 [anon]
10a000-112000 rw-p 00000000 00:00 0 [anon]
3fe0a000-40609000 rwxp 00000000 00:00 0 [anon]
40609000-4060a000 rwxp 00000000 00:00 0 [anon]
/ #
```

0x40_0000_0000 USER_TOP

0x3e_0000_0000 + ASLR 偏移
动态解释器加载地址

中间空洞由 mmap/mremap 填入

用户栈顶
0x4000_0000 + 随机偏移($\leq 8\text{MiB}$)

argc / argv / envp /
auxv / AT_RANDOM / strings

brk 用户堆起始位置
在最高 LOAD 末尾后 roundup 位置

0x0001_0000 ET_DYN/static-PIE 加载固定偏移量
ELF LOAD 段 + BSS

0x0

未来计划

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

► 完善多核启动路径和调试

- 第二版内核4月起构建，目前多核尚不稳定

► 设备树支持与硬件驱动生态

- 发挥异步模型优势加速外设开发

► 开发板适配

- 从 QEMU 迈向生产级环境

► RCU路径迁移与性能提升

- RCU机原型已经实现，但尚未完全部署
- 目前与上游队伍以性能差距为主，分差仅100余分

AI使用与经验

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

- **知识学习与设计。**借助AI快速上手往届优秀作品与一些开源的内核项目。
- **想法验证与原型搭建。**在假期学习阶段，我们用AI参考BSD搭建了第一版内核，阅读和修改原型给我们提供了宝贵的参考经验。
- **内核调试。**AI对反汇编工具链有良好的知识，在裸机开发环境帮助我们节省了大量时间。我们为AI准备了一系列xtask命令，封装了常见的反汇编、qemu内存dump与栈追踪、scause decode工具，并准备了相关的debug skill辅助AI调试。
- **性能调优。**我们用AI构建了tx-observe框架，在系统调用关键节点输出带时间戳和层级关系log，导出为parquet格式供AI用sql分析热点路径，避免AI低效读取或通过打印调试浪费token。



design	5 Jun 2026 at 8:37 PM
05_filesystem	18 Jun 2026 at 7:49 PM
04_process-signals	18 Jun 2026 at 7:49 PM
01_substrate	18 Jun 2026 at 7:49 PM
00_meta-framework	18 Jun 2026 at 7:49 PM
INDEX.md	5 Jun 2026 at 8:37 PM
03_memory-vm	5 Jun 2026 at 8:37 PM
02_execution	5 Jun 2026 at 7:11 PM
06_devices	12 May 2026 at 4:25 PM

和 AI 讨论设计文档

The useful duration counters in this run are VM recipe counters:

Counter	Count	Total	p50	p99	Max
debug.vm.recipe.publish.duration_ns	5,006	466.436 ms	91 us	197 us	1.075 ms
debug.vm.recipe.reclaim_tree.duration_ns	5,010	171.452 ms	37 us	77 us	465 us

Related non-duration VM counter:

Counter	Count	Total	p50	p99	Max
debug.vm.recipe.publish.touched_entries	5,006	22,115	4	7	15

This supports the syscall-level picture: VM mapping and unmapping work is a real part of pthread lifecycle cost, but the larger repeated buckets are still the syscall spans around clone, munmap, futex, sigprocmask, mmap, and exit.

AI利用 tx-observe 生成的 profile 报告

AI使用与经验

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

- **项目的知识管理。** AI生成代码的速度远大于人阅读的速度，没有架构知识很快就会跟不上AI生成代码的思路，所谓 code is cheap, show me the talk.
- **避免AI“走捷径”。** 我们主要通过两个思路克服：一是善用linter等代码检查工具，拒绝编译含有旧接口的代码；二是跟上代码库的知识，避免对AI提出模糊的指令。
- **工具赋能AI。** AI输出上下文有限，不可能装入代码库的所有知识。以性能调优为例，代码量大后AI不可能跟踪整个调用流程，而flamegraph等追踪框架能快速帮AI定位关键路径，能极大节约AI开发时间。
- **Harness工程同样重要：** 过多的skills、mcp注入会污染和浪费Agent的上下文。应当追求知识和技能对Agent的渐进式披露(progressive disclosure)，给设计文档和skills建立简单的索引，让AI按需读取，而不是盲目探索或全部注入。

总结

完成情况

系统介绍

未来计划

AI使用声明

总结

TxKernel

► 我们的TxKernel内核:

- **功能完备**: 覆盖进程、线程、VM、VFS、ext4/FAT、网络、TTY、IPC, 已实现 243 条系统调用。
- **面向现实应用**: 支持 BusyBox、vi、TCC、SQLite 等真实用户态程序, 而不是只跑手写测例。
- **架构有特色**: 异步无栈协程 + Mailbox + StepOutcome, 统一建模“订阅、阻塞、唤醒”。
- **工程可演进**: Identity/Payload 分离、Zone/EBR 生命周期管理、tx-observe 性能观测和 xtask 自动化工具链。



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

CSCC 2026

敬请指教！

TxKernel开发团队

队员：杨岩琰、刘佳硕、孟书培

指导教师：夏文、仇洁婷

TxKernel